WILLIAM & MARY

CHARTERED 1693

CSCI 455/555: GenAI for SD

Spring 2026

Prof. Antonio Mastropaolo

Assistant: Md. Zahidul Haque Alvi

Summarizing code via LSTM models

Your goal is to implement an LSTM model that can generate natural language summaries for Java methods.

Specifics You are asked to implement an LSTM for code summarization. This involves taking decisions on various points:

- **Corpus Construction**: The language is fixed *i.e.,* Java (method-level). You must **construct your own training and validation data** by mining public GitHub repositories. As a reference for how such datasets are built, see the CodeXGLUE code-to-text dataset [1] on Hugging Face. Your focus is on **Java** methods.

  Requirements:

  - ~**50,000** code–summary pairs for training.
  - A separate validation set (1000 sample).
  - Document sources and pre-processing in your notebook.

- **Model Architecture**: LSTM, following the same structure from the class notebook. Adapt the bug-fixing model so that the decoder generates natural language tokens instead of code tokens.

- **Embeddings & Vocabulary**: You will be using the provided `get_codet5_embeddings.py` script to tokenize your data and extract the pretrained embedding matrix.

  Steps:

  1. Prepare your data as `.txt` files (one sample per line). Flatten each Java method into a single whitespace-normalized line. Lowercase each summary.
  2. Install dependencies in Colab:

     ```
     !pip install -r requirements.txt
     ```

  3. Run the provided script to tokenize your data and extract the pretrained embedding matrix:

     ```
     !python get_codet5_embeddings.py --input sample_code.txt \
         --output train_code.pt
     !python get_codet5_embeddings.py --input sample_summary.txt \
         --output train_summary.pt --max_length 128
     ```

     You are responsible for choosing an appropriate `max_length` for your data (up to a maximum of 512, which is CodeT5+'s limit). To verify your setup, you can first test with the provided sample files (`sample_code.txt` and `sample_summary.txt`) in the shared Drive folder.

4. Load the output in your notebook and plug it into your LSTM:

```
data = torch.load("train_code.pt")
embedding_layer = nn.Embedding.from_pretrained(
    data['embedding_matrix'], freeze=False)
token_ids = data['token_ids']
```

The script outputs a `.pt` file containing the token IDs for your data and the CodeT5+ pretrained embedding matrix (32,100 × 768). This means your LSTM starts with embeddings that already understand code – no vocabulary building needed. In your notebook, load and use it as follows:

```
import torch
import torch.nn as nn

data = torch.load("train_code.pt")

# plug directly into your LSTM
embedding_layer = nn.Embedding.from_pretrained(
    data['embedding_matrix'], freeze=False)

# feed into the LSTM
token_ids = data['token_ids']

# Useful constants
pad_id = data['pad_token_id']
eos_id = data['eos_token_id']
vocab_size = data['vocab_size']
embedding_dim = data['embedding_dim']
```

- **Evaluation**: Compute the following metrics on the provided test set:
  - BLEU-1,2,3,4
    Use a standard implementation: `https://github.com/mjpost/sacrebleu`
  - METEOR
  - BERTScore
  - SIDE
    Use the original implementation provided in the following repository.

  To note: Use early stopping based on BLEU-1 computed on the validation set to select the best performing checkpoint. A checkpoint is a saved snapshot of the model's parameters (and optionally optimizer state) at a given epoch/step during training; you should keep the checkpoint with the highest validation BLEU-1 and stop training when BLEU-1 does not improve for a fixed patience (e.g., 3 consecutive evaluations).

- **Test:** You will evaluate your final (best) model on a **held-out test set of 1,000 Java code–summary pairs provided by the instructor**. **Do not** use this test set for training, validation, hyperparameter tuning, or early stopping; it must be used only once for final reporting of the metrics listed above.

  All required files – including the test dataset, the embedding script, sample files, and `requirements.txt` – are available in the shared Google Drive folder.

Submission Instructions

Deadline: Friday March 15th, 2026 @ 11:59 PM EST.

Your submission must include a link to your repository including:

- A Jupyter notebook (`assignment-2-LSTM.ipynb`) containing all code, outputs, and write-up. Runnable top-to-bottom.
- A short `README.md` describing:
  - data source(s) and pre-processing,
  - how to install dependencies and reproduce,
  - where outputs are written.

Grading Rubric

Component	**Points**	**Details**
Dataset Preparation	25	Sources documented, ~50K pairs, proper splitting
Model & Training	25	Working LSTM, early stopping, loss convergence
Avoid Overfitting	10	Early stopping, train vs. validation loss analysis
Evaluation Metrics	25	All metrics reported (BLEU, METEOR, BERTScore, SIDE)
Generation of Predictions	15	Model generates summaries on test set
Total	**100**	

References

[1] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664*, 2021.